Data Models and Module Diagrams

Elijah, Ziya, Kiran, Aryaman

Manager: Trent

Recap

A mobile app that welcomes you to campus, introducing you to all of the exciting club events and announcements.

Stay tuned in to student organizations you follow and receive notifications of upcoming important events. All events you save will go to your personal calendar to help you stay on top of the hectic business of campus life.

This app will help connect students to student organizations, getting everyone involved!

Tech Stack

1

Front End

React Native (Expo):
Cross-platform iOS & Android from a single codebase. Enables faster iteration and code sharing across platforms with a large ecosystem of libraries.

2

Back End

Node and Express:
JavaScript across the full stack reduces context switching. Express is lightweight and unopinionated, giving us flexibility as requirements evolve.

3

Database

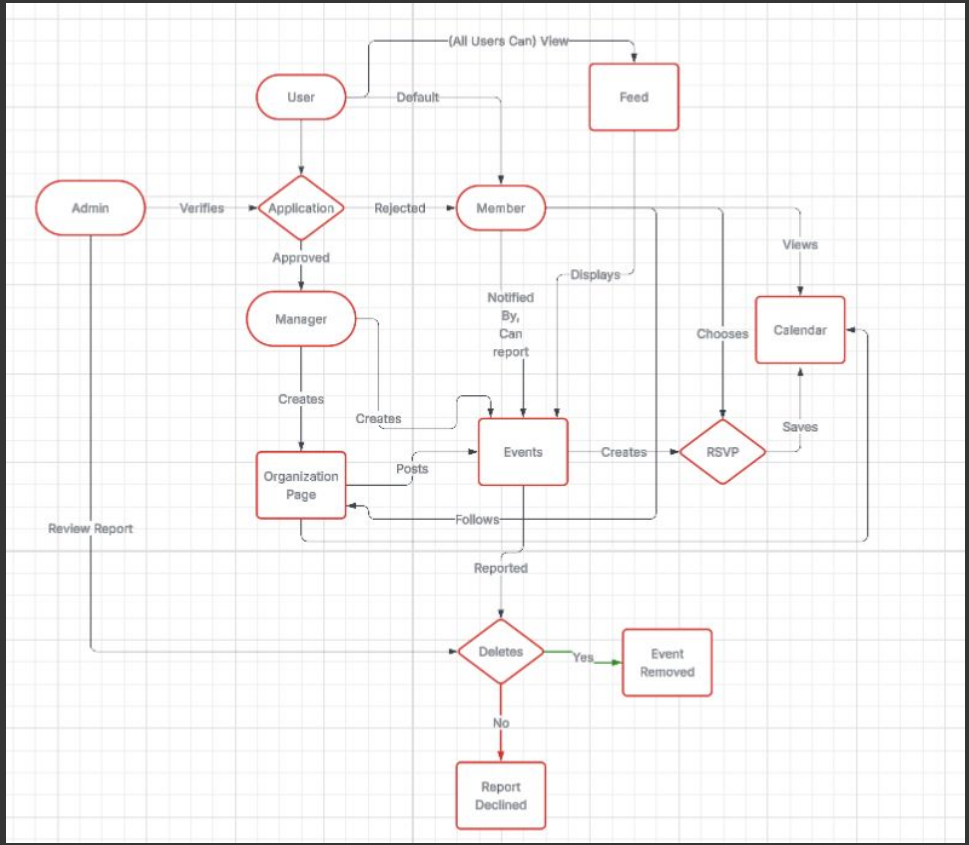
PostgreSQL:
Relational database with excellent support for complex queries, ACID compliance, and strong community support for production workloads.
Hosted on Neon Serverless Postgres

4

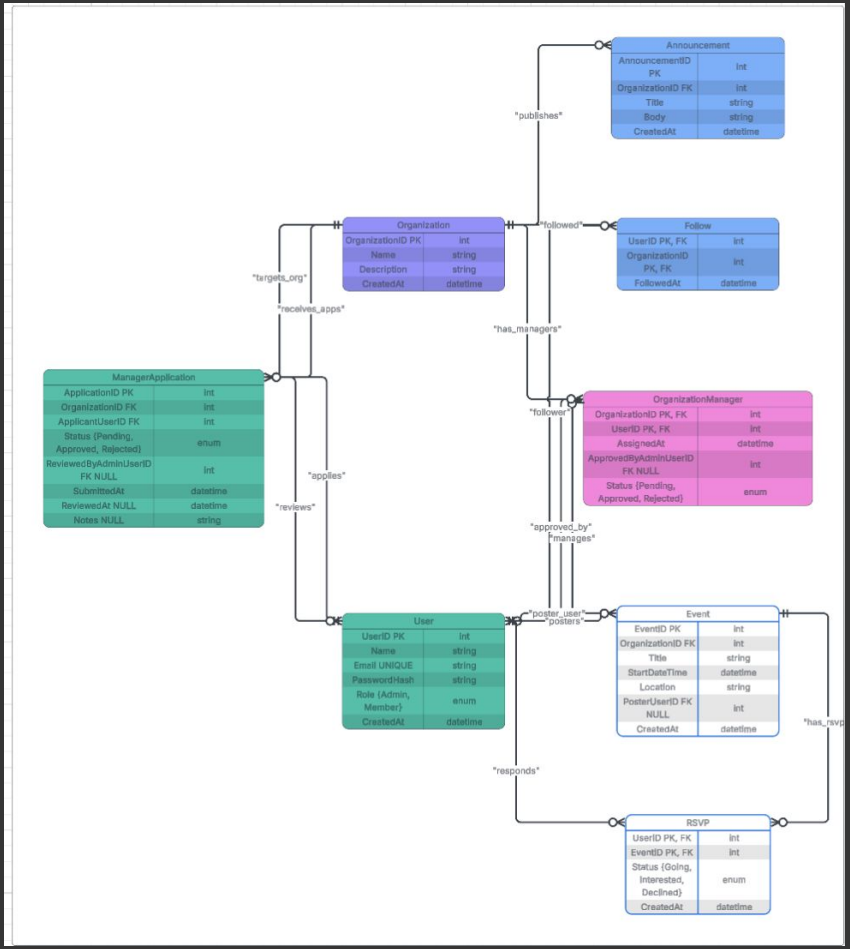
ORM

Prisma ORM:
Type-safe database access with auto-generated queries. Simplifies migrations and schema changes, reducing bugs from raw SQL and boosting developer velocity.

Data Model | Diagram



ER Diagram


Announcement
AnnouncementID PK int
OrganizationID FK int
Title string
Body string
CreatedAt datetime
"publishes"
Organization
OrganizationID PK int
Name string
Description string
CreatedAt datetime
"followed"
Follow
UserID PK, FK int
OrganizationID PK, FK int
FollowedAt datetime
"targets_org"
"receives_apps"
"has_managers"
ManagerApplication
ApplicationID PK int
OrganizationID FK int
ApplicantUserID FK int
Status {Pending, Approved, Rejected} enum
ReviewedByAdminUserID FK NULL int
SubmittedAt datetime
ReviewedAt NULL datetime
Notes NULL string
"follower"
OrganizationManager
OrganizationID PK, FK int
UserID PK, FK int
AssignedAt datetime
ApprovedByAdminUserID FK NULL int
Status {Pending, Approved, Rejected} enum
"applies"
"reviews"
"approved_by"
"manages"
User
UserID PK int
Name string
Email UNIQUE string
PasswordHash string
Role {Admin, Member} enum
CreatedAt datetime
"poster_user"
"posters"
Event
EventID PK int
OrganizationID FK int
Title string
StartDateTime datetime
Location string
PosterUserID FK NULL int
CreatedAt datetime
"has_rsvp
"responds"
RSVP
UserID PK, FK int
EventID PK, FK int
Status {Going, Interested, Declined} enum
CreatedAt datetime

End Points & Function Signature

Auth

1

POST localhost/auth/signup

- user.create(email: string, password: string, name: string)
- user.create(email: string, password: string, name: string) : AuthToken | null

2

POST localhost/auth/login

- user.login(email: string, password: string)
- user.login(email: string, password: string) : AuthToken | null

End Points & Function Signature

Feed

1

GET localhost/feed/:userID

- retrieve all events from organizations the user follows
- feed.getByUserID(userID: int)
- feed.getByUserID(userID: int): Event []

End Points & Function Signature

Organizations

1

GET localhost/organization/listAll

- will retrieve all organizations
- organization.listAll()
- organization.listAll(): Organization []

2

GET localhost/organization/listOne/:id

- will retrieve the organization with id == {:id}
- organization.listOneByID(id: int)
- organization.listOneByID(id: int): Organization

3

POST localhost/organization/create

- create an organization
- organization.create(managerID: int, name: string)
- organization.create(managerID: int, name: string): boolean

End Points & Function Signature

Organizations

4

GET
localhost/organization/listAnnouncements/:id

- will list all announcements by a specific organization
- organization.listAnnouncementsByID(id: int)
- organization.listAnnouncementsByID(id: int): Announcement []

5

POST
localhost/organization/addAnnouncement

- organization.addAnnouncement(managerID: int, orgID: int, body: string)
- organization.addAnnouncement(managerID: int, orgID: int, body: string): Announcement

End Points & Function Signature

Follow

1

POST localhost/follow/create/:userID/:orgID

- will follow an organization
- follow.create(userID: int, orgID: int)
- follow.create(userID: int, orgID: int): boolean

2

DELETE localhost/follow/delete/:userID/:orgID

- will unfollow an organization
- follow.delete(userID: int, orgID: int)
- follow.delete(userID: int, orgID: int): boolean

3

GET localhost/follow/listAll/:userID

- Lists all organizations followed by a user
- follow.listAllByUserID(userID: int)
- follow.listAllByUserID(userID: int): Organization[]

End Points & Function Signature

Events

1

GET localhost/event/listAll

- event.listAll()
- event.listAll(): Event []

2

GET localhost/event/listOne/:id

- will show event with that specific ID
- event.listAllByID(int: id)
- event.listAllByID(int: id): Event

3

GET localhost/event/listByOrg/:orgID

- events by a specific organization ID
- event.listAllByOrgID(orgID: int)
- event.listAllByOrgID(orgID: int): Event []

End Points & Function Signature

Events

4

POST localhost/event/create

- event.create(managerID: int, orgID: int, title: string, date: DateTime)
- event.create(managerID: int, orgID: int, title: string, date: DateTime): Event

5

DELETE localhost/event/delete/:id

- will delete the event with id == {:id}
- event.deleteByID(id: int)
- event.deleteByID(id: int): Boolean

End Points & Function Signature

Manager Applications

1

POST localhost/application/create

- will submit a manager application
- application.create(userID: int, orgID: int, message: string)
- application.create(userID: int, orgID: int, message: string): void

2

GET localhost/application/listAll

- will show all manager applications
- application.listAll()
- application.listAll(): Application []

3

POST localhost/application/review/:id

- approves a specific application based on ID
- application.review(adminID: int, applicationID: int, action: "approve" | "reject"): boolean

Testing Strategies

1 Unit Testing

- Test individual backend functions to ensure correct logic. Functions like **user.create()**, **event.create()**, and **organization.listAll()**.

2 API Testing

- Verify Express endpoints return correct responses. For example,

POST /auth/signup

Request Body:

```
{
  "email": "test@example.com",
  "password": "123456",
  "name": "Test User"
}
```

3 Integration Testing

- Ensure frontend communicates correctly with backend. Example flow:

React Native App → Express API → PostgreSQL

Testing Examples

1

Test: organization.create()

Call organization.create(managerID: 1, name: "Chess Club")
Expected: Returns true
Verify: Call organization.listAll() or check PostgreSQL with PGAdmin

2

Test: event.create()

User fills event form in React Native app → clicks "Create". Triggers POST to /event/create
Expected: New event row in PostgreSQL
Verify: Navigate to events list or query event.listByOrg(5)

3

Test: user.login()

Setup: Populate users table with test account
Call: user.login("test@test.com", "password123")
Expected: Returns AuthToken object
Verify: Token is valid and can access protected routes

More Testing Examples

1

Test: follow.create()

Call: follow.create(userID: 1, orgID: 5)
Expected: Returns true
Verify: Call follow.listAllByUserID(1) and confirm org 5 appears

2

Test: application.review()

Setup: Create test application with application.create(userID: 3, orgID: 2, message: "I want to manage this club")
Call: application.review(adminID: 1, applicationID: 1, action: "approve")
Expected: Returns true
Verify: Check user 3 now has manager privileges for org 2 in PostgreSQL

3

Test: feed.getByUserID()

Setup: User 1 follows orgs 3 and 5, both have events
Call: feed.getByUserID(1)
Expected: Returns Event[] with only events from orgs 3 and 5
Verify: Confirm no events from non-followed orgs appear in the result

Thank you